Project Report
(Task Manager)

Course: B.Tech CSE (AI&ML)

Student Name: Mohit Nagar

Registration Number: 25BAI11366

Project Type: Bring Your Own Project (BYOP)

Faculty Name: Dr. Suneet Joshi

Subject Name: Fundamentals in AI & ML

Introduction & Problem Statement

As a first-year B.Tech student, I observed that the transition to college life brings a significant increase in the volume of daily responsibilities, including lab records, club assignments, and academic deadlines. I found that "mental clutter"—trying to keep track of these tasks without a dedicated system—often leads to stress and decreased productivity.

The problem I chose to address is the lack of a simple, distraction-free environment for task logging. While many mobile applications exist, they are often bloated with advertisements or complex interfaces that distract from the primary goal: getting things done. My project, the **Simple Task Manager**, provides a focused Command-Line Interface (CLI) to solve this issue.

Why This Project Matters

Productivity is built on organization. This project is significant because:

- **Cognitive Offloading:** By externalizing tasks into a digital list, the user can focus their mental energy on execution rather than memorization.
- **Progress Visualization:** Seeing a "Completed" list provides psychological reinforcement and motivation to continue working.
- **Accessibility:** A CLI tool is lightweight, requires zero internet connectivity, and can run on any system that supports Python, making it a universal utility for students.

Methodology & Approach

My approach was to design a system that mimics the natural human workflow of handling chores: **Identification → Action → Verification.**

```
Menu:
1. Add a task
2. Mark a task as done
3. Show all tasks
4. Exit

Enter your choice (1-4):
```

I broke the solution down into four logical modules:

1. **Data Architecture:** I utilized two global Python lists, `todo_list` and `done_list`, to maintain the state of tasks throughout the session.
2. **Functional Decomposition:** I avoided writing one long script and instead created dedicated functions for `add_task()`, `done_task()`, and `show()`. This makes the code modular and easier to debug.
3. **The Control Loop:** I implemented a `while True` loop that keeps the program active, allowing the user to perform multiple operations without having to restart the script.
4. **UI/UX Design:** Even within a text-based terminal, I focused on user experience by using clear prompts and formatted output to distinguish between pending and finished tasks.

Key Technical Decisions

- **Batch Processing Logic:** To improve efficiency, I designed the `add_task` function to handle multiple entries at once. By asking the user for a count `n`, the program stays in a sub-loop until all tasks are entered, this is much faster than returning to the main menu for every single item.

```
Enter your choice (1-4): 1

--- Adding Tasks ---
How many tasks do you want to add?
Enter number: 2
Enter task: Homework
Enter task: Workout
Tasks added successfully!
```

- **Membership Verification:** A critical decision was to use the `in` keyword to verify task existence. This ensures that if a user tries to mark a task as "done" that doesn't exist, the program provides a helpful error message instead of failing silently.
- **Visual Separators:** I used string repetition (`"=" * 20`) to create a clear visual hierarchy in the terminal. This simple design choice makes the output look like a structured report rather than a wall of text.

Challenges Faced & Troubleshooting

- **The List Mutation Challenge:** In my early drafts, I attempted to remove items from a list while iterating through it. This led to "index skipping," where Python would miss tasks. I resolved this by moving away from index-based loops and using direct string matching and the `.remove()` method, which proved much more stable.
- **Input Integrity:** I realized that users might accidentally enter empty strings. I addressed this by ensuring the input is captured effectively, and I explored how to keep the program running even when a user enters an invalid menu choice.
- **Empty State Management:** Initially, the program looked "broken" when no tasks were present because it printed nothing. I improved the `show()` function by adding `if len(...) == 0:` checks to provide the user with encouraging messages like "No tasks pending."

```
Enter your choice (1-4): 3

====================
YOUR TO-DO LIST:
- Homework
- Workout

YOUR COMPLETED TASKS:
- (No tasks completed yet)
====================
```

Future Scope

While this version satisfies the course requirements, I have identified several areas for future growth:

- **Data Persistence:** Adding file handling (`.txt` or `.csv`) to save tasks so they remain available after the program is closed.
- **Priority Levels:** Tagging tasks as "High," "Medium," or "Low" to help users prioritize their work.
- **Deadline Tracking:** Integrating a date/time module to alert users about upcoming deadlines.

Conclusion & Key Learnings

This project served as a comprehensive application of the Python fundamentals taught in this course. I learned how to decompose a real-world problem into logical functions, manage data structures effectively, and prioritize user experience in software design. It has given me the confidence to build more complex tools as I progress through my B.Tech journey.

Appendix

```
# Task Manager Project
# Using: Lists, Functions, While Loops, and Conditionals

todo_list = []
done_list = []

def add_task():
    """Adds a specific number of tasks to the to-do list"""
    print("\n--- Adding Tasks ---")
    print("How many tasks do you want to add?")
    n = int(input("Enter number: "))

    while n > 0:
        task = input("Enter task: ")
        todo_list.append(task)
        n = n - 1
    print("Tasks added successfully!")

def done_task():
    """Moves a task from the to-do list to the done list"""
    print("\n--- Mark Task as Done ---")
    s = input("Enter the task you have completed: ")

    # Using 'in' to check if it exists before removing
    if s in todo_list:
        todo_list.remove(s)
        done_list.append(s)
        print("Great job! Task moved to Completed.")
    else:
        print("Error: That task is not in your list.")

def show():
    """Prints both lists clearly"""
    print("\n" + "="*20)
    print("YOUR TO-DO LIST:")
    if len(todo_list) == 0:
        print("- (No tasks pending)")
    else:
        for task in todo_list:
            print("- " + task)

    print("\nYOUR COMPLETED TASKS:")
    if len(done_list) == 0:
```

```
        print("- (No tasks completed yet)")
    else:
        for task in done_list:
            print("- " + task)
    print("="*20 + "\n")

def exits():
    """Final summary before closing the program"""
    print("\nThank you for using Task Manager!")
    show()
    exit()

# Main Program Loop
while True:
    print("Menu:")
    print("1. Add a task")
    print("2. Mark a task as done")
    print("3. Show all tasks")
    print("4. Exit")

    choice = int(input("\nEnter your choice (1-4): "))

    if choice == 1:
        add_task()
    elif choice == 2:
        done_task()
    elif choice == 3:
        show()
    elif choice == 4:
        exits()
    else:
        print("Invalid choice! Please try again.")
```